

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

**QUICK CMS IMPLEMENTATION ENGINE WITH
SAILSJS AND REACTJS**

by

Pham Quang Khang

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

© Copyright by Pham Quang Khang 2021

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

**QUICK CMS IMPLEMENTATION ENGINE WITH
SAILSJS AND REACTJS**

by

Pham Quang Khang

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:
Vu Thu Diep

Quick CMS Implementation Engine With SailsJS and ReactJS

Pham Quang Khang

Degree Master of Software Engineering

FPT University

2021

Abstract

The role of information systems is essential for a business, especially those operating in technology. The information system works with people, information technology, and processes to accomplish business objectives. Now a day, a lot of businesses need an enterprise's internal administration application. They need the main functions rather than beautiful interfaces. And what is the solution to get the functionality out as quickly as possible to meet the business needs?

When discussing the functionality of a website, we will often get the answer that it will be done in the backend - the business processing center of a website. However, based on the reality of evaluating many projects, we can see that the effort spent on developing the backend only about 20% of the total time; the remaining 80% is spent on frontend development. That's why we built a CMS engine to cut down on the development time of an enterprise's internal administration application. In this article, we will clarify the system architecture that we have built up to now, which has proved to be an outstanding advantage in supporting CMS and giving someone permission extremely quickly, ensuring the safety of data, give it to the right person with authority

keywords: CMS, Content Management System, SailsJS, ReactJS, Front-end templating, restful API, JWT, Redis, MySQL

Acknowledgments

I would first like to thank my family, parents, brother, sister & brother-in-law, and nephews for all their love and support over the years. I would also like to thank my lover for all her help, motivation, patience.

I want to express my sincere gratitude to my advisor Dr. Vu Thu Diep, for the continuous support of my Master's study and research. With his guidance, I did have the opportunity to attend this Master's course. His patience, motivation, and immense knowledge helped me all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my Master's study.

My sincere thanks also go to couch Nguyen Hien, Dr. Phan Duy Hung, Dr. Nguyen Thi Bich Thuy, Dr. Tran Trong Hieu, Dr. Pham Quang Nhat Minh, Dr. Nguyen Thi Viet Huong, and Dr. Nguyen Nghi Thanh for all their given valuable knowledge. Without their equipment, I could not qualify for this thesis.

I would also thank my collages in FPT Software, Media-One for all their support and facilitation for all my studying time. It helped me to overcome the stiffness of working and studying.

Last but not least, I would like to thank all of my friends, the RTC team, academic support team in the Master's program of FPT University. I appreciate all their support, advice in life, and the educational environment.

Table of Contents

Contents

Acknowledgments.....	2
Table of Contents.....	3
List of Tables	6
List of Figures	7
Chapter 1 Introduction	1
1.1 Reason.....	1
1.2 Review	2
1.3 Research Purpose Statement	2
Chapter 2 System Requirement	4
2 4	
2.1 Architecture overview.....	4
2.1.1 Frontend: <i>Includes built-in fixed modules that provide a user interface that helps developers define display elements and grant permissions to end-users.....</i>	5
2.1.2 Backend: <i>App's data storage and processing center</i>	6
2.2 System requirements.....	7
Chapter 3 System design and implementation.....	8
3 8	
3.1 System Deployment Architecture	8
3.2 Backend.....	9
3.2.1 About SailsJS Framework.....	9
3.2.2 The remarkable point in the backend of this engine, what will be gained by using this template?.....	9
3.2.3 Auth module (Authentication and Authorization module)	10
3.2.4 Captcha module	11

3.2.5	ORM with Blueprint APIs	12
3.2.6	User management APIs.....	12
3.2.7	Page management APIs	13
3.2.8	App Configuration APIs	13
3.2.9	Cache.....	13
3.2.10	Database	13
3.3	Frontend	14
3.3.1	Overview	14
3.3.2	Login module	16
3.3.3	Form Viewer module	17
3.3.4	List Viewer module.....	18
3.3.5	Navbar Viewer module	19
3.3.6	Page Definition Module	20
3.3.7	View Component templates.....	22
3.3.8	Cache.....	23
3.4	Communication.....	23
3.4.1	Between Client to server	23
3.4.2	Between server nodes in cluster mode.....	23
3.5	Implementation techniques	25
Chapter 4	Contribution	26
4	26	
4.1	Contribution in technology	26
4.2	Contribution in management.....	27
Chapter 5	Conclusion and perspectives	28
5	28	
References or Bibliography		29

Bibliography 29

List of Tables

Table 1: Distribution of forms type in 5 WCMS projects from 2019 to 2021 *) <i>Statistical data of 5 WCMS projects we worked on from 2019 - 2021</i>	1
Table 2: Functional Requirement.....	7
Table 3: Contribution in technology	27

List of Figures

Figure 1: Architecture overview of engine	4
Figure 2: System Deployment Architecture	8
Figure 3: Authentication APIs	11
Figure 4: Main layout.....	15
Figure 5: Login form.....	16
Figure 6: Form Viewer sunk in container	17
Figure 7: Form Viewer in Popups.....	18
Figure 8: List Viewer sunk in container	18
Figure 9: List Viewer in popups	19
Figure 10: 2-tier menu and menu dynamic configuration page	20
Figure 11: Configure information for the form interface.....	21
Figure 12: Configure information for the button using in form or list viewer.....	21
Figure 13: Configure information for the APIs using in page	22
Figure 14: Configure information for the list/table interface.....	22
Figure 15: Local Storage caching information	23
Figure 16: Communication	24

Chapter 1

Introduction

1.1 Reason

Organizations typically need to document work and use their resources to work on documents. Documents are essential for legal compliance [20], process improvement, training, and customer support. Moreover, access to the content in corporate documents enables enriched information sharing and collaboration and better decision making. So, building a content management system (CMS) is always an essential need of today's businesses. CMS is deployed to integrate business processes, reduce work processing time, transacting information faster, improving financial management, making process knowledge clear; it helps to convert data to knowledge (challenges in enterprise management) [1].

With the practice of developing many web internal management systems, greater than 70% of CMS's data interaction interface is in data table/table form (list of business records) and input form (form for entering data or review a piece of record information).

Form Type	Total screen	Rate (%)
Table	83	~37
Input	122	~54.5
Dashboard	5	~2.2
Chart	2	~0.89
Comment	1	~0.45
Calendar	1	~0.45
Other type	10	~4.5
Total	224	100

Table 1: Distribution of forms type in 5 WCMS projects from 2019 to 2021

**) Statistical data of 5 WCMS projects we worked on from 2019 - 2021.*

Based on that statistic and analysis (Table.1), the CMS Engine helps programmers create interfaces and perform functions on the built-in interfaces without touching the frontend code for these two main types of user interfaces built.

1.2 Review

The content management system is a software system that provides preservation, organization, and dissemination services for digital collections [4]. With the proliferation of Web content and the growing demand of customers for web-based services, companies have an increasing need to support their Web content with Web content management systems. As a result, current estimates indicate that the market for content management systems is growing at a rate of over 29% per year worldwide [3].

Because of the rapid growth and open market, there are many products related to the construction and development of CMS, such as WordPress CMS [19], Joomla CMS [22], Drupal CMS [23], October CMS [24], Django CMS [25], Laravel Framework [26], Jhipster Framework [8]... They make developing a CMS much more straightforward and save time and money in different ways. For example, it builds on top of plugin architectures like WordPress or using extensions base like Joomla, or with Jhipster is the power of code-gen technique using Yeoman-based generator [7] [9] ...

All CMS or Framework mentioned above are very famous and have millions of developers or web professionals using them every day. To match the business process, it all requires a lot of customization. They all prefer or are limited to generating source code for developers to edit or add their functionality. This limitation sometimes costs you a lot of time to code, develop, especially UI development.

1.3 Research Purpose Statement

For this thesis, I want to introduce "Quick CMS Implementation Engine with SailsJS and ReactJS", the solution to define and generate the primary management interface of a CMS site using JSON schema for configuration. Using this engine, programmers can quickly build CMS and build interfaces with logic to interact with data by defining them right on the admin interface. The results of the solution can be applied to building several

existing applications that have been put into production use in some businesses in Vietnam (Fsoft, Cocobay)

Chapter 2 System Requirement

2.1 Architecture overview

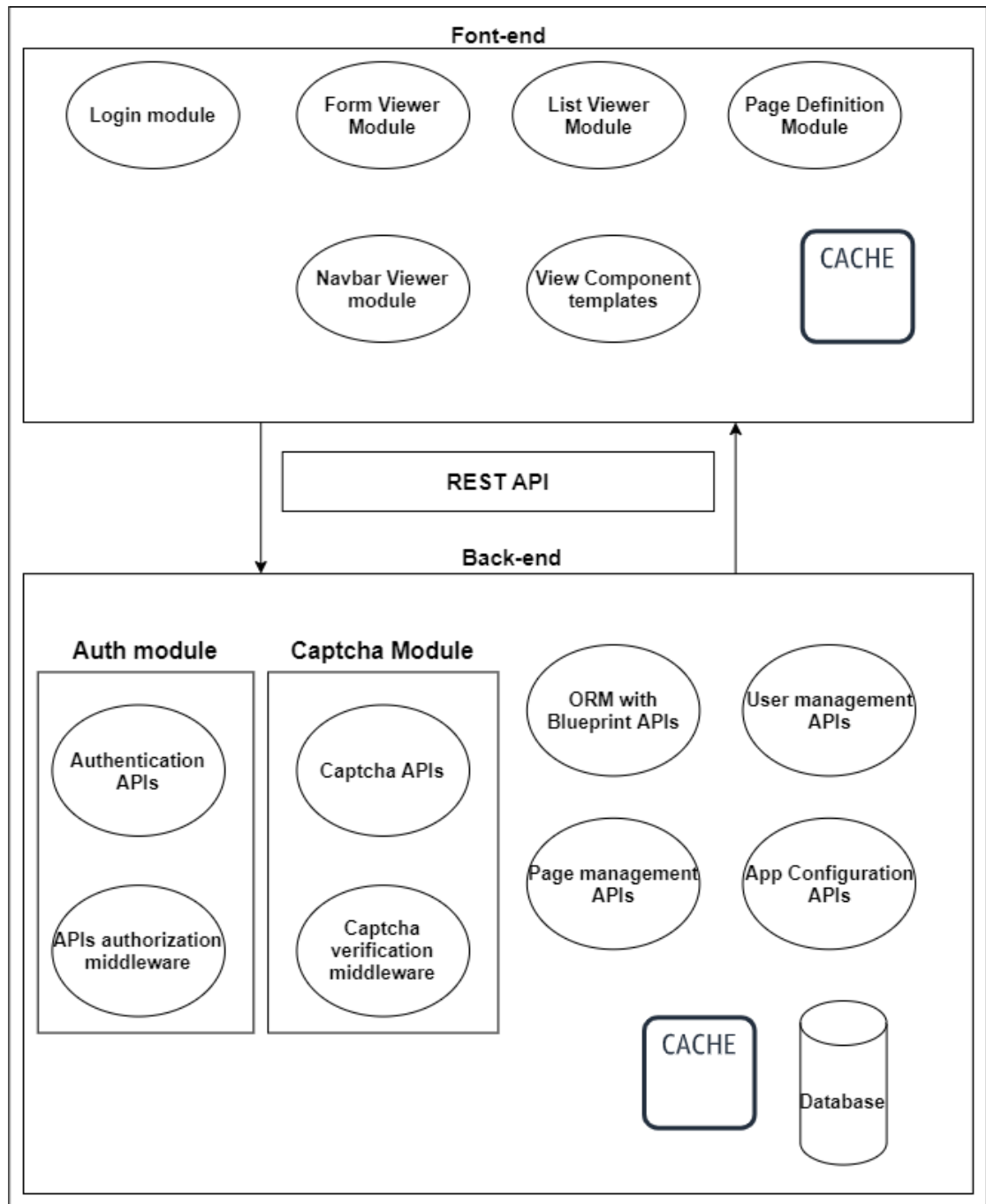


Figure 1: Architecture overview of engine

2.1.1 Frontend: *Includes built-in fixed modules that provide a user interface that helps developers define display elements and grant permissions to end-users.*

- Login module: Module used to authenticate users using the service
- Page Definition module: The page is responsible for installing interface configuration, logic, APIs for components displayed on a Form viewer or a List viewer.
- Form Viewer Module: The module responsible for rendering the form interactively with data records (CRUD) according to the interface definition configuration obtained from the Page Definition module
- List Viewer Module: The module responsible for displaying tabular data according to the interface definition configuration obtained from the Page Definition module
- View component templates: Small interface elements (e.g., input text, input key reference etc.) that pre-installed with display, data entry, data validation... waiting for the developer to choose to arrange into the view to match the data and data type needs to interact
- Navbar viewer module: Module that reads menu configuration information to help decentralize menu display permissions based on the current authorization of the logged-in user
- Cache: Local storage [10] stores the current user's login session information

2.1.2 Backend: *App's data storage and processing center*

- Authentication APIs: Set of APIs to authenticate and issue user tokens for later API calls
- APIs authorization middleware: Check the validity of the user token as well as the user's right to interact with the API is valid or not
- Page management APIs: Set of APIs used for adding, updating, and deleting business site configurations.
- ORM with Blueprint APIs: Set of CRUD APIs is automatically generated after defining the data mapping model with the Database (Only available when the API definition goes through the APIs authorization middleware).
- User management APIs: Set of APIs to help manage system user information as well as user permissions.
- App Configuration APIs: Set of APIs to help manage system configuration settings information
- Captcha APIs: Captcha code generation APIs for anti-automated robots
- Captcha Verification Middleware: Check the validity of the captcha code (if any) when calling a specific API of the system
- Database: Stores some default initialization configurations of the engine, providing default functions for developers such as User management, authorization management, navbar management, configuration information management shape of system, View system log...
- Cache: A temporary set of data that is queried frequently to improve backend responsiveness

2.2 System requirements

This engine provides an interface that defines page information and the right to interact with the page to complete the backend API and define the interface suitable with the API provided by the backend.

The functional requirements of the engine are described in the table below

User Permission	Ordinal number	Content
System Admin	1	Manage the list of interface pages: Interface definition, permissions, actions to interact with data
	2	Manage permission and user role groups: Define the rights users will use in the system
	3	User management: manage the list of users allowed to access the system
	4	View system log
	5	Manage system configuration information
	6	System API management: API definition and API access
Other Permissions	1	Pages are developed and authorized by the system admin to allow action

Table 2: Functional Requirement

Chapter 3

System design and implementation

3.1 System Deployment Architecture

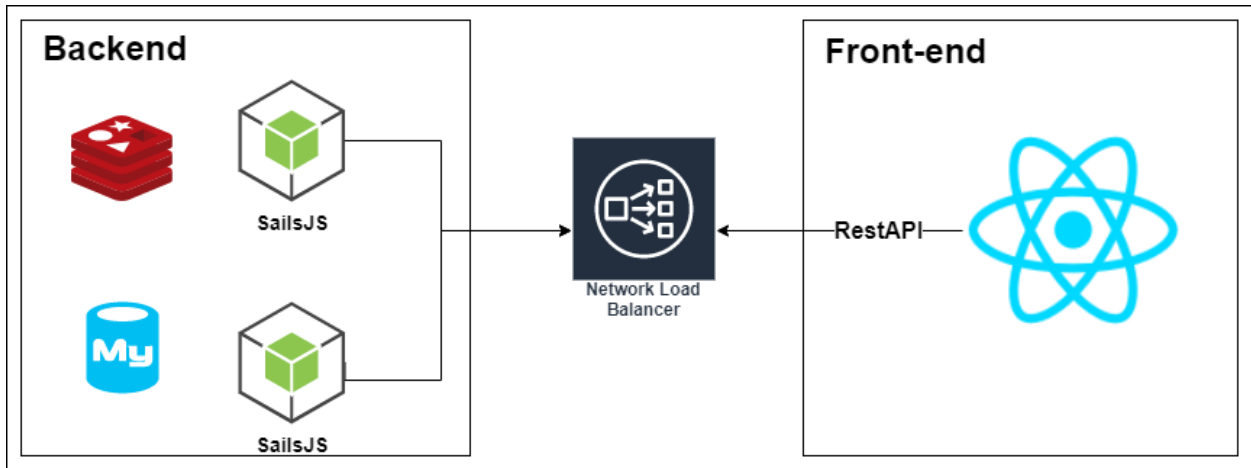


Figure 2: System Deployment Architecture

- The engine is built entirely in the Javascript programming language with a backend based on the SailsJS [11] framework running Nodejs [12] and a frontend built by ReactJS [13] to make it easy for developers to write new components for the template set.
- The engine uses Redis [14] as cache and implements a pub-sub mechanism to synchronize configuration information of backend nodes to help the engine fully support system load distribution through Network Load Balancer
- The engine uses MySQL as the central database to store data, an extremely famous open-source database management system. In addition, MySQL offers stability and responsiveness both in terms of performance and scaling, so it is also a widely used database management system in today's commercial applications.

3.2 Backend

3.2.1 About SailsJS Framework

- **Sails** is the most popular MVC framework for NodeJS, designed to emulate the familiar MVC pattern of frameworks like Ruby on Rails, but with support for the requirements of modern apps: data-driven APIs with scalable, service-oriented architecture.
- **Sails** is a lightweight framework that sits on top of Express [21]. Its ensemble of small modules works together to provide simplicity, maintainability, and structural conventions to NodeJS apps.
- **Auto-generated REST API:** Sails comes with blueprints that help jumpstart your app's backend without writing any code.
- **Any database:** Sails bundles a powerful ORM, Waterline, which provides a simple data access layer that works, no matter what database you are using.

3.2.2 The remarkable point in the backend of this engine, what will be gained by using this template?

- Using Sails JS is a clean, well-structured framework with easy-to-find documentation and highly accessible to use.
- Built-in system user management module
- Built-in module for user login and access control
- Built-in dynamic grant permission mechanism for individual users or a group of system users
- Blueprint API and support for dynamic configuration APIs (Grant permission, request captcha, configure return data etc.)
- Support mechanism to help scale horizontal applications quickly, meet a large amount of load to the system when needed

3.2.3 Auth module (Authentication and Authorization module)

Like many other CMS, login, and authorization is an essential feature. This is the first function when the user authenticates and is granted access to the system's authorized information. And with this engine, we have built-in the Authentication and Authorization module, which includes APIs used for giving login permissions to users and middleware that checks the user's API access every time a request is sent to the server.

➤ **Authentication APIs**

This engine uses and implements stateless authentication with JSON Web Token (JWT) [15] and built a mechanism for dynamically configuring the time to live (TTL) of the issued Token, as well as there is a mechanism to renew the Token when the user operates with the system. In addition, list API provides user authentication functionality including:

- *Login API (/api/auth/account):* API collects system user information, including account name, password, captcha, then make a query to check user information and provides bearer JWT to use for the following API calls
- *API refresh token (/api/auth/refresh-token):* API used to get a new token to replace the old Token for token renewal

The API description diagram is shown in figure 3 below.

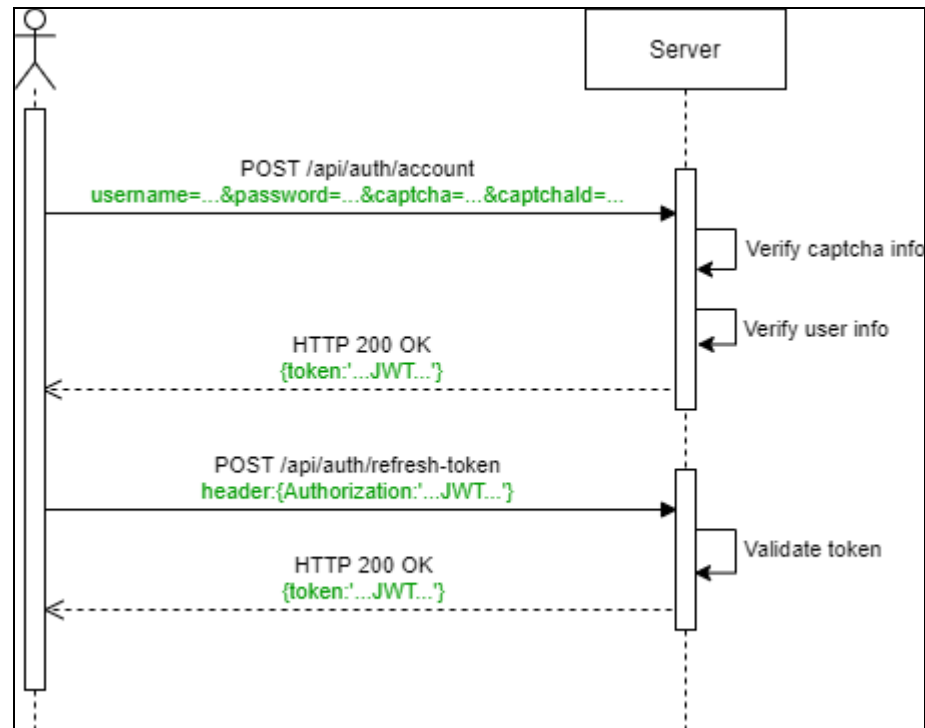


Figure 3: Authentication APIs

➤ **APIs authorization middleware**

SailsJS allows developers to define Policies as middleware between the Client and the server that helps to apply one or more sets of rules to one or more external backend APIs. And the engine has pre-configured several regulations to check the availability of requests to all the APIs that the backend provides.

As mentioned in the previous section, the engine uses stateless authentication, so the issued JWT will be attached on each request to the server to identify the user as well as control user access after login. Authorization middleware is the module written to decrypt the Token, get the user information from the Token, and check the user's access to the API being called. Thus, authorization middleware is the first gateway to every request from the Client to API Endpoint.

3.2.4 Captcha module

Like the APIs authorization middleware mentioned above, Captcha verification middleware is also the Policy that every request from the client-side calling to the API

Endpoint will have to go through. This middleware is responsible for checking the captcha information provided by the user (Including captcha ID and captcha text) whether it exists in the system and is valid. When the provided captcha code is correct, the request is passed to the endpoint that processes the request and returns the response according to the processing results at the API. And in the Captcha APIs module is the API set that provides and generates Captcha code images for the Client to decode.

3.2.5 ORM with Blueprint APIs

- *ORM (Object Relational Mapping):*[16] is a programming technique that helps to map data records in the database management system to the type of objects defined in classes - a common concept used in all modern languages today
- *Blueprint APIs:* SailsJS is designed to minimize the time and the number of lines of code that must be written to create an API. Blueprints are the way Sails uses to generate APIs based on system design.
- *ORM with Blueprint APIs:* This is a combination of the above two items. Sails provide users with a quick CRUD API set after defining the complete mapping data model with the database. For example, when you define a user mapping model with a user table in the database, you will immediately have a RESTful API for CRUD manipulation with user table data.

3.2.6 User management APIs

The set of APIs helps the system administrator to manage the list of users in the system. The basic features that the API set provides to administrators can be mentioned as:

- Update user data information
- Update user permissions
- Add new user
- Only interact with authorized users

3.2.7 Page management APIs

The engine is implemented to store the frontend user interface definition configuration as JSON Schema. Each Schema corresponding to a screen or a page will be stored in the database. It is entirely possible to change the view component using at client-side based on updating the JSON Schema. This set of APIs is used to perform CRUD interaction to help system administrators interact with the corresponding JSON Schema pages.

3.2.8 App Configuration APIs

When installing the application, there are inevitably many parameters that need to be configured dynamically (For example, SMTP info etc.). It will be configured according to the running environment. This set of APIs helps users define more configuration parameters to use or update configuration parameters used during system operation. Parameters will be stored back in the database. They will also be loaded into the system cache to be called and used quickly and efficiently in offloading queries to the database.

3.2.9 Cache

Using Redis-cache, the engine stores' backend frequently accessed and rarely changed records at the cache to reduce query load to the database.

In addition, Redis is also used as a pub-sub channel [14] to help communicate and synchronize backend nodes when running apps under cluster mode.

3.2.10 Database

MySQL [18] is the database management system used in the engine. The interface is generated from reading the JSON schema stored in the database, so the template comes with a built-in database backup of several system information configuration pages.

3.3 Frontend

3.3.1 Overview

➤ **ReactJS**

- The front-end is built using the ReactJS framework, one of the most powerful frameworks currently used to create website interfaces.
- ReactJS Build with **component base** - Build encapsulated components that manage their state, then compose them to make complex UIs.
- Since component logic is written in JavaScript instead of templates, developers can easily pass rich data through the app and keep the state out of the DOM.

➤ **CoreUI React Admin Template (CoreUI)**

The front end of the template is built on the **CoreUI React Admin Template** - the admin template is used by a large community of programmers for quick deployment of the admin page. Some features of CoreUI:

- Save thousands of development hours by using ready-made, reusable, and commonly used widgets and UI components. Moreover, CoreUI React Admin Template is delivered with a ready-to-use environment, so you don't have to waste time configuring the project. All build scripts and tools are ready to use out of the box.
- CoreUI originates from open-source.
- No design skills required: CoreUI is based on CoreUI Components Library with beautifully handcrafted UI Components designed by a team of experienced designers.
- UI Components and Widgets are written with readability in mind, and they are comparatively short and easy to understand
- Responsive, Mobile & Cross-Browser Compatibility

➤ Main Layout

The website's layout is arranged into four circled areas and numbered from 1 to 4, as shown below.

- Zone 1: *Header*, including website logo and hidden sidebar button
- Zone 2: *Header*, where to place the system's feature buttons, shortcuts
- Zone 3: *SideBar*, Menu navigation to the pages of the website
- Zone 4: The main container, which contains the business processing views etc.

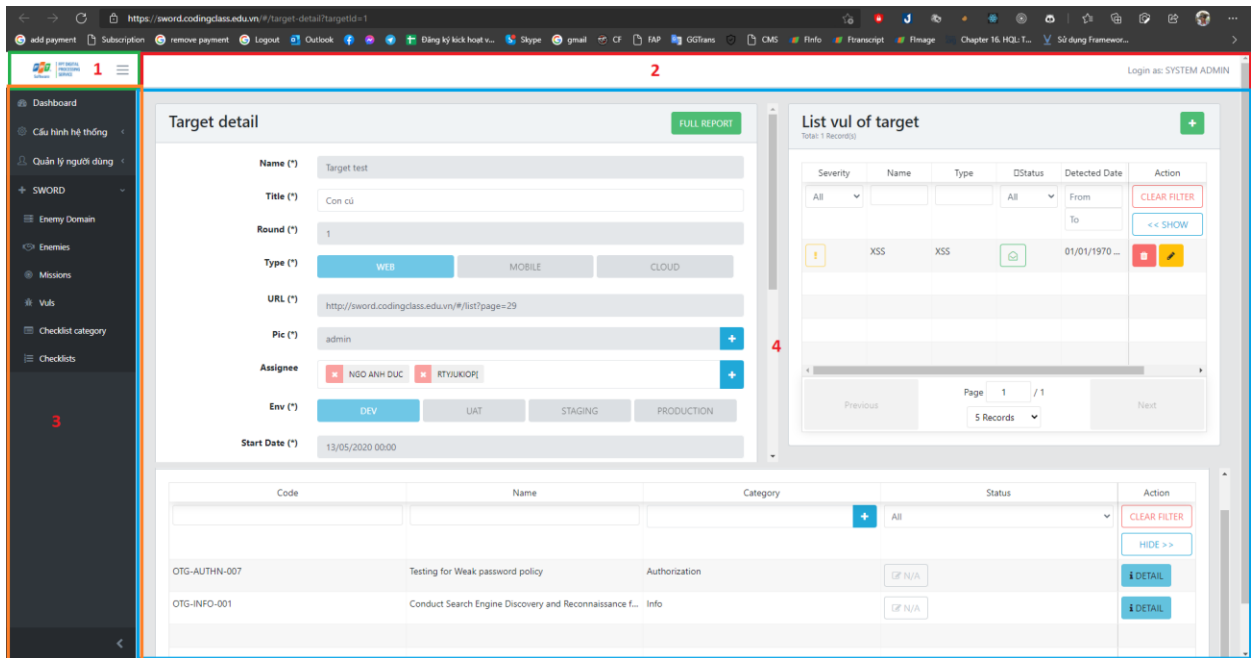
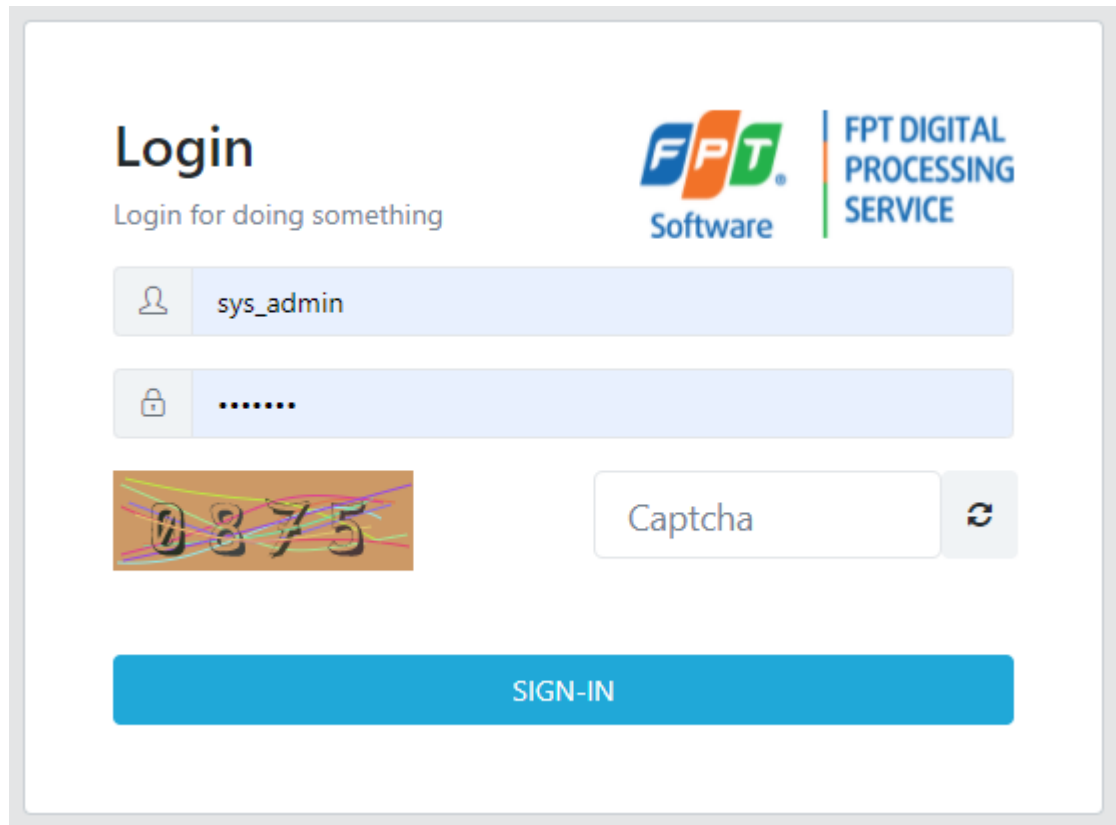


Figure 4: Main layout

3.3.2 Login module

The login module of the frontend is built and integrated with the provided backend authentication module. The built-in module provides a login interface for users with two sets of parameters required to perform login and account anti-detection by the robot.

- Login account information: Username and Password
- User decoded captcha information: `captchaId` and `captchaText`



The screenshot displays a login interface for 'FPT Digital Processing Service'. At the top left, the word 'Login' is prominently displayed in a large, bold, black font, with the tagline 'Login for doing something' in a smaller font directly below it. To the right of the text, the FPT logo (stylized 'FPT' in blue and orange) is shown above the word 'Software' in blue. Further to the right, the text 'FPT DIGITAL PROCESSING SERVICE' is displayed in blue, separated from the logo by a vertical line. The form itself consists of three main input areas: a username field with a person icon on the left and the text 'sys_admin' inside; a password field with a lock icon on the left and a series of dots '.....' inside; and a captcha section featuring a small image of a captcha with the numbers '0875' and a text input field to its right labeled 'Captcha'. A small circular refresh icon is located to the right of the captcha text field. At the bottom of the form, a large, solid blue button with the text 'SIGN-IN' in white capital letters is centered.

Figure 5: Login form

3.3.3 Form Viewer module

The screenshot displays a web application interface for generating tokens. The main content area is titled "Gen token". It contains several input fields: "ID tài khoản người dùng (*)" with a blue "+" button, "Thời gian sống token (phút - 0=Không giới hạn)" with a value of "0", and "Email nhận token". Below these is a "Thông tin thêm" section with a red background and a green checkmark. At the bottom, there is a "Mã xác nhận (*)" field with a CAPTCHA image showing the number "2511" and an "input" button. A red button labeled "TAO TOKEN" is at the bottom right. The left sidebar shows a menu with options like "Dashboard", "Cấu hình hệ thống", "Quản lý menu", "Quản lý trang", "Thông tin cấu hình", "Quản lý quyền", "Quản lý nhóm quyền", "Quản lý api public", "Tạo token người dùng", "Log call api", "Log Email", "Quản lý người dùng", "SWORD", "Enemy Domain", "Enemies", "Missions", "Vuls", "Checklist category", and "Checklists". The top right corner shows "Login as: SYSTEM ADMIN".

Figure 6: Form Viewer sunk in container

- The module reads the JSON schema information and builds the user interface of the input form.
- Based on the settings of the JSON schema, the module creates a container containing the View Component templates and executes the predefined logic according to the JSON schema definition. (Validate data, embed external data, etc.).
- This module is mainly used to view detailed information, update or create the new data record.

- This engine provides two types of forms that share the same core of the form viewer module (Fig.6 and Fig.7)

Figure 7: Form Viewer in Popups

3.3.4 List Viewer module

ID	Full name	Phone number	Email	Gender	Grant type	Description	Block status	Lần đầu đăng ...	Tạo bởi	Ngôn ngữ	Action
5	Demo account	0909909909	demo@codin...	Male	Lord		All	01/01/1970 0...	1	vi	UPDATE CHANGE PASSWORD ACCOUNT
4	rtjyjiop[	qertyuio	1234ttrfd@h...	Other	Warrior		All	22/04/2020 1...	3	vi	UPDATE CHANGE PASSWORD ACCOUNT
3	Ngo Anh Duc		ducna@soft.c...	Male	Lord			17/04/2020 0...	1	vi	UPDATE CHANGE PASSWORD ACCOUNT
2	admin	0981604050	khanguq.vn@...	Male	Lord			17/04/2020 0...	1	vi	UPDATE CHANGE PASSWORD ACCOUNT

Figure 8: List Viewer sunk in container

- List viewer module reads the JSON schema information and builds the user interface in the form of a table list
- Based on the settings of the JSON schema, the module generates a data table with user-defined data fields to display.
- Unlike the form viewer, the list viewer is mainly used to display tabular data and provide filters and search for data on the data table. All data loading, data pagination, filters, sorters etc. are all built-in with Blueprint API Interface, so it's speedy to get the interface after the user completes the API set.
- The system provides two types of forms that share the same core of the list viewer module (Fig.8 and Fig.9)

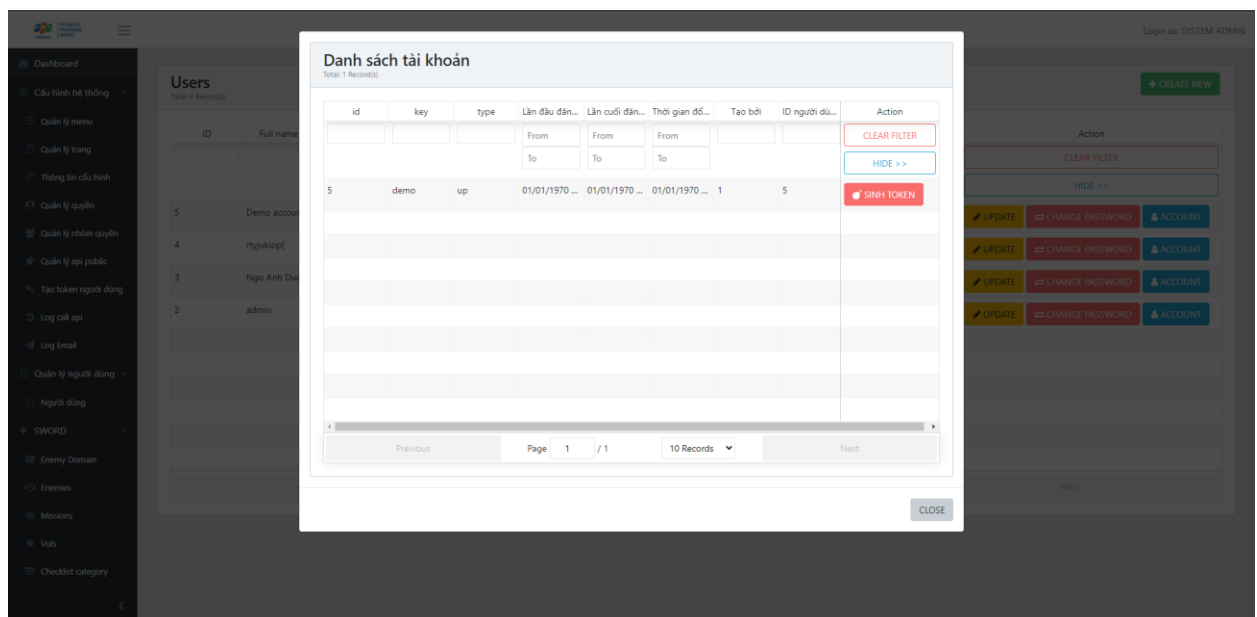


Figure 9: List Viewer in popups

3.3.5 Navbar Viewer module

This module is responsible for reading the menu configuration information stored in the database to display a shortcut to the target page on the left corner navbar.

The system provides a 2-tier hierarchical menu and provides an interface to define menu information for developers to help developers manage menus and dynamically assign permissions for menus displayed to each user object.

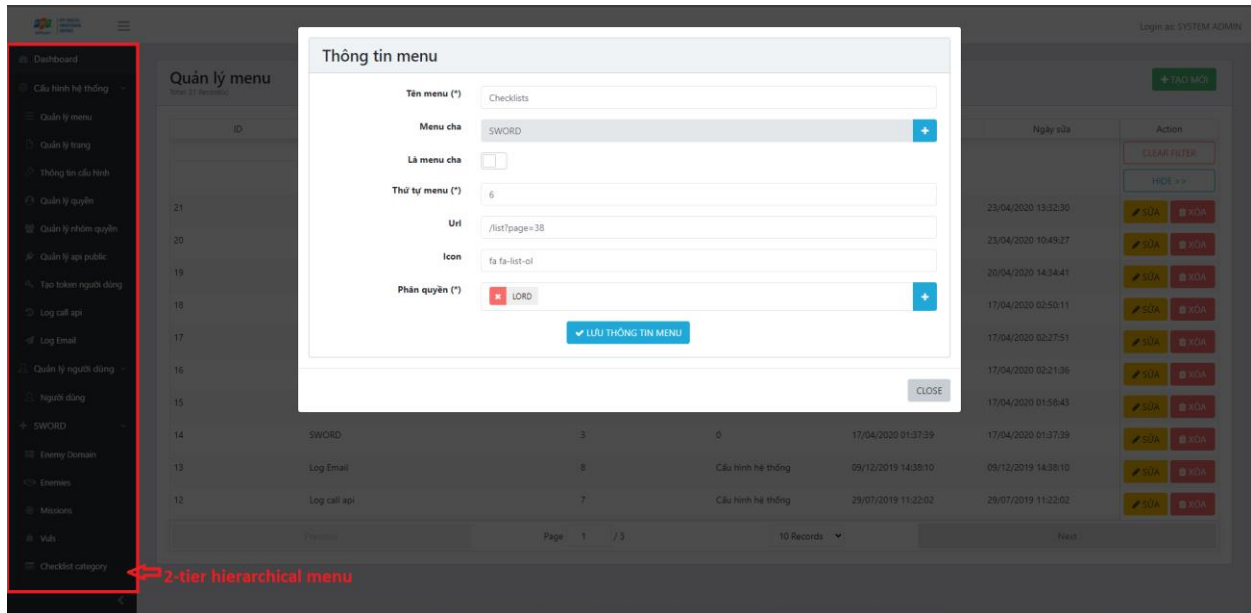


Figure 10: 2-tier menu and menu dynamic configuration page

3.3.6 Page Definition Module

The module helps developers manage the system's list of pages. In addition, the engine provides a user interface to help developers create JSON schema that defines the interface of the system pages.

The module provides an interface with four primary tabs (Form, Button, Api, Grid); each tab allows the developer to define a type of website parameter.

- Form: Tab is used to configure information for the form interface, used for the form viewer module (Fig.11)
- Button: Configure the information of the buttons on the form viewer or list viewer (Fig.12)
- API: Configure information about the APIs used on the page (Fig.13)

- Grid: Configure information for the tabular interface used in the List viewer module (Fig.14)

Sửa thông tin trang

Tên trang: Target vul detail | Mô tả: thông tin chi tiết về cập nhật vul của target | Hàm tải dữ liệu: find | Ngôn ngữ: | Phân quyền: BASIC

Form | Button | API | Grid | Form JSON

Form
Quản lý form

Thuộc tính	Giá trị
Type *	Type
Name *	type
Description	Placeholder
Observation	
Risk score *	
Risk score *	
Detected date *	
Influence Object	
Evidences	
Remediation	

Figure 11: Configure information for the form interface

Sửa thông tin trang

Tên trang: List vul of target | Mô tả: List vul of target | Hàm tải dữ liệu: find | Ngôn ngữ: | Phân quyền: BASIC

Form | Button | API | Grid | Form JSON

Button
Quản lý các nút bấm trong form

Thuộc tính	Giá trị
Chế độ	edit
Tên	New vul
Phân quyền	
Màu nút	success
Outline	
Biểu tượng	fa fa-plus
Hiển thị trên cột	
Điều kiện ẩn	[{"this.id": [{"in": [{"id": "10", "user.userId": "10"}, {">": "10"}, {"<": "user.id", ">": "1", "<": "1"}, {"user.id": {"<": "1", "id": "10"}]}], [{"id": "10", "user.userId": "10"}, {">": "10"}, {"<": "user.id", ">": "1", "<": "1"}, {"user.id": {"<": "1", "id": "10"}]}]
Kích hoạt	Form Popup
Dữ liệu nhúng	[{"page": "47", "mode": "create", "embed": [{"target": "#target*"}]}]
Xác nhận	
Quay lại sau khi submit	

Figure 12: Configure information for the button using in form or list viewer

Sửa thông tin trang

Tên trang: List vul of target | Mô tả: List vul of target | Hàm tải dữ liệu: find | Ngôn ngữ: | Phân quyền: BASIC

Form | Button | API | Grid | Form JSON

API

Quản lý API

API: find, delete

Thuộc tính

Tên	find
Kiểu	find
Uri	/api/target/vul
Mô tả api	Lấy thông tin vul của target
Phương thức http	GET
Đi qua captcha	☐
Phân quyền	BASIC
Tùy chọn gửi lên(update data)	string
Tùy chọn cập nhật(where)	number
Dữ liệu gửi lên	isDelete
Dữ liệu hạn chế gửi lên (restrict field)	0
Dữ liệu trả về	
Bool Expression	Biểu thức tính quyền truy cập api theo thông tin người dùng và dữ liệu gửi lên

Figure 13: Configure information for the APIs using in page

Sửa thông tin trang

Tên trang: List vul of target | Mô tả: List vul of target | Hàm tải dữ liệu: find | Ngôn ngữ: | Phân quyền: BASIC

Form | Button | API | Grid | Form JSON

Grid

Quản lý hiển thị bảng dữ liệu

Grid: Severity

Thuộc tính

Tên	Severity
Trường dữ liệu	severity
Điều kiện ẩn	
Phân quyền	
Kiểu dữ liệu	number
Format number	☐
Danh sách có sẵn	number
Các lựa chọn	None, Low, Medium, High, Critical

Figure 14: Configure information for the list/table interface

3.3.7 View Component templates

React components are code written by developers with the need to reuse many times in many places. These are the components that will be defined and used in the form viewer and the list viewer.

The engine has built-in a wide range of component templates that are sufficient for the basic needs of data entry and a data display.

3.3.8 Cache

The engine uses local storage to store some static information to reduce data querying from the backend or database.

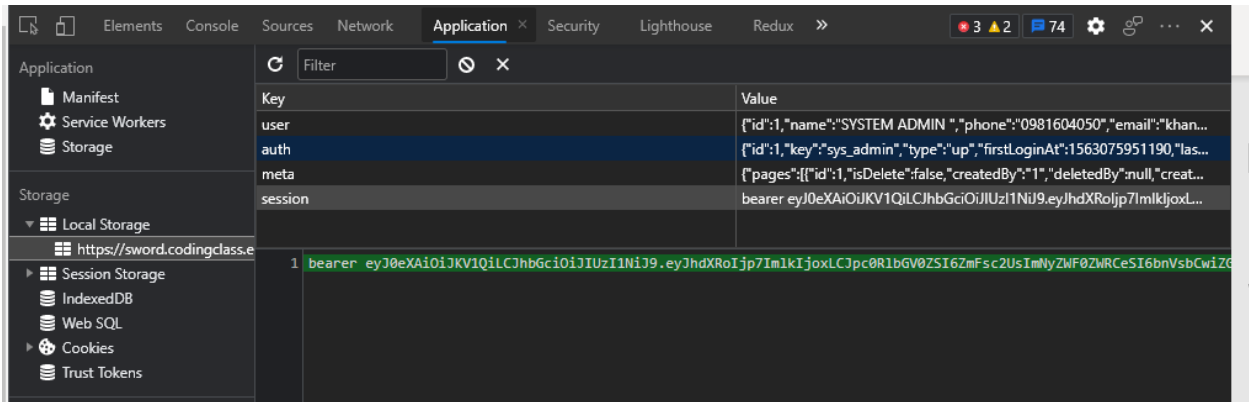


Figure 15: Local Storage caching information

3.4 Communication

3.4.1 Between Client to server

- *HTTP request standard RESTful*: almost all requests from Client to server are made by HTTP request under standard RESTful API design.
- *Socket*: Another communication channel provided by the backend allows the Client to connect to the server continuously and respond to pub-sub tasks.

3.4.2 Between server nodes in cluster mode

This engine is configured with Redis pub-sub to synchronize tasks and cache data between nodes running the cluster. Redis pub-sub allows one node in the cluster network to broadcast an event. The remaining nodes immediately receive the event notification to perform information updates at the base node to help the system synchronize as quickly as possible.

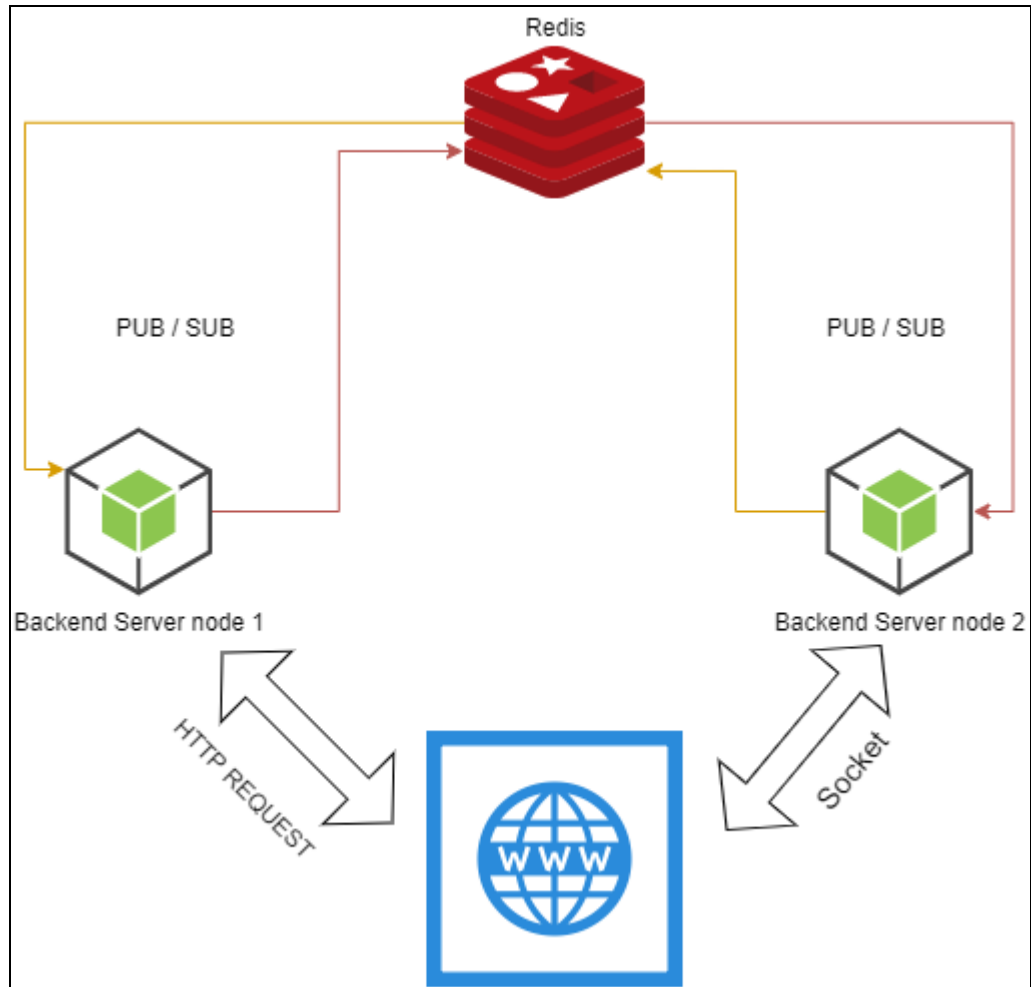


Figure 16: Communication

3.5 Implementation techniques

The engine was introduced to minimize the development time of the user interface as well as helping to effectively decentralize data, so the technical part of the application's implementation is also reduced a lot of stages in the process of Analyze, evaluate and develop the interface. Specifically, when using the template set, we need to perform the following steps:

1. Analyze the function to be built
2. Component selection, the conceptualization of functional interfaces, and evaluation of the feasibility of functional implementation with pre-built components
3. Design Database, API backend suitable for the function
4. API Deployment and Testing
5. Define the JSON schema for the functional interface to be built at the interface of the Page Definition Module

With these technical steps, we have almost skipped the interface design steps and only focused on building user functions, helping to reduce the time and effort of application development. In addition, by using the JSON schema that defines the display data fields, the engine provides "runtime user customization" which means that at the time of operation, the user can easily update the interface to add/remove the displayed field without the developer having to update the code

Chapter 4 Contribution

4.1 Contribution in technology

Currently, on the market, many frameworks help speed up the programming process to help reduce application development time in general in many different ways, as mentioned in the previous sections. However, in general, these frameworks will all generate code so that users will customize the generated code. Furthermore, as said from the beginning, with the traditional way, 80% of application development time is building user interfaces, so frameworks are aimed at minimizing this development time. And below will be the assessment and comparison between the two ways of the two frameworks in reducing the time to build the user interface.

	This Engine	Other
Approach	pre-built components and perform action definition for components based on JSON schema with defined structure	Code generation, predefined code structure for users to customize on the generated code code
Advantages	- Quickly create user interfaces without touching the UI code. The programmer generates the interface by defining the JSON schema - Since there is no need to touch the UI code, there are almost no UI bugs - Programmers do not need to know the UI code can also use	Easily customize the user interface according to the purpose with the knowledge of user interface programming

Disadvantages	- Programmers don't have much choice for interface customization - The needs that the engine does not support will have to build additional component code	- Requires programmer to know user interface programming - Bugs can be generated during the editing process, which takes time a lot
---------------	---	--

Table 3: Contribution in technology

4.2 Contribution in management

- The engine helps to reduce application development time by 60-70% as well as save 2/3 man/month and speeding up the time of putting applications into production deployment
- The engine helps reduce data risks when it has built a set of API gateways that require grant permission on each API provided to the outside.
- Some applications using this engine are being applied at FPT software and statistics:
 - **Order Management System:**
 Total number of screens: ~40 screens
 Real Efforts with engine: 30 man/day
 Estimate Efforts: 2→4 man/day/page => ~80 →160 man/day
 Save Efforts: 60 → 80%
 - **SWORD:** Information management system for penetration testing
 Total number of screens: ~30 screens
 Real Efforts with engine: 20 man/day
 Estimate Efforts: 2→4 man/day/page => ~60 →120 man/day
 Save Efforts: 65 → 83%

Chapter 5

Conclusion and perspectives

Thus, in this thesis, I propose an engine with significant advantages to help speed up the business development process by quickly creating applications that support business administration with high data security and data security. As a result, save both in terms of cost as well as time to build the application.

The solution to actual projects in many enterprises in Vietnam is reducing real development time and effort from 60 to 80% compared to the expectation. In Section 4.2, there is a concrete demonstration of the advantages of the engine

This Engine is perfectly suitable for internal business administration applications that need to quickly develop and secure internal company data to help speed up the production process and develop the company in a short time.

In the future, the engine will continue to be developed with more components to help solve more different problems of interface processing. It will soon release a specific release version with detailed documentation. Let's check out the pre-release version at [khang21081995/Sails-CMS-Template \(github.com\)](https://github.com/khang21081995/Sails-CMS-Template)

References or Bibliography

Bibliography

- [1] S. N. C Benevolo, "Evaluation of Content Management Systems (CMS): a Supply Analysis," *Electronic Journal of Information Systems Evaluation*, vol. 10, no. 1, pp. 9-22, 2007.
- [2] E. K. Ritu Agarwal, "Time Flies When You're Having Fun: Cognitive Absorption and Beliefs about Information Technology Usage," *MIS Quarterly*, vol. 24, pp. 665-694, 2000.
- [3] S. McKeever, "Understanding Web content management systems: evolution, lifecycle and market," *Industrial Management & Data Systems*, vol. 103, no. 9, pp. 686-692, 2003.
- [4] Y. Han, "Digital content management: the search for a content management system," *Library Hi Tech*, vol. 22, no. 4, pp. 355-365, 2004.
- [5] J. C. M. d. Igor Lima, Practical Detection of CMS Plugin Conflicts in Large Plugin Sets, Elsevier B.V, 2019.
- [6] W. Y. Cao Xiang, "Using Content Management System Joomla! to build a website for research institute needs," in *International Conference on Management and Service Science*, Wuhan, China, 2010.
- [7] "Yeoman," [Online]. Available: <https://yeoman.io/>. [Accessed 5 6 2021].
- [8] "Jhipster," [Online]. Available: <https://www.jhipster.tech/>. [Accessed 5 6 2021].
- [9] K. S. Prasad Reddy, "Introducing JHipster," in *Beginning Spring Boot 2*, 2017, pp. 279-287.
- [10] W. W. a. S. M. Pulimood, "ANALYSIS OF PRIVACY AND SECURITY IN HTML5 WEB STORAGE," *The Journal of Computing Sciences in Colleges*, vol. 27, no. 3, pp. 80-87, 2012.

- [11] "SailsJS," [Online]. Available: <https://sailsjs.com/>. [Accessed 6 6 2021].
- [12] "NodeJS," [Online]. Available: <https://nodejs.org/en/>. [Accessed 6 6 2021].
- [13] "ReactJS," [Online]. Available: <https://reactjs.org/>. [Accessed 6 6 2021].
- [14] "Redis," [Online]. Available: <https://redis.io/>. [Accessed 6 6 2021].
- [15] B. C. C. M. Michael B. Jones, "JSON Web Token (JWT) Bearer Token Profiles for OAuth 2.0," 2013.
- [16] M. Keith and M. Schnicariol, "Object-Relational Mapping," in *Pro JPA 2*, 2010, p. 69–106.
- [17] H. T. MD Da Silva, *Redis Essentials*, Birmingham, UK: Packt Publishing, Ltd, 2015.
- [18] "MySQL," [Online]. Available: <https://www.mysql.com/>. [Accessed 6 6 2021].
- [19] "wordpress," [Online]. Available: <https://wordpress.com/>. [Accessed 6 6 2021].
- [20] J. E. Scott, "User Perceptions of an Enterprise Content Management System," in *44th Hawaii International Conference on System Sciences*, Hawaii, 2011.
- [21] "ExpressJS," [Online]. Available: <https://expressjs.com/>. [Accessed 6 6 2021].
- [22] "joomla," [Online]. Available: <https://www.joomla.org/>. [Accessed 6 6 2021].
- [23] "drupal," [Online]. Available: <https://www.drupal.org/>. [Accessed 6 6 2021].
- [24] "octobercms," [Online]. Available: <https://octobercms.com/>. [Accessed 6 6 2021].
- [25] "django-cms," [Online]. Available: <https://www.django-cms.org/en/>. [Accessed 6 6 2021].
- [26] "laravel," [Online]. Available: <https://laravel.com/>. [Accessed 6 6 2021].